

Code-Carbon: Full Project Walkthrough

A Modular, Publication-Oriented Research Framework for Predictive Carbon-Aware Software Engineering

EXECUTIVE SUMMARY:

Code-Carbon is a modular, publication-oriented research framework designed to analyze Python source code, detect energy inefficiencies (*energy smells*), estimate runtime carbon footprints early in the software development life cycle (SDLC), perform safe automated AST code refactoring, and benchmark empirical execution in isolated sandboxed subprocesses. It integrates static AST analysis, Radon complexity metrics, hardware power profiling (RAPL), live marginal carbon grid intensity querying via the Electricity Maps API v4, fuzzy belief math engines (ESS & CIRS), a FastAPI backend, and an HSL glassmorphism dark-mode React dashboard.

1. System Features & Key Capabilities

- **Static AST Analysis:** Comprehensive parsing of function calls, loops, complexity indices, file/network operations, and recursion behaviors without executing the source file.
- **Energy Knowledge Base (EKB):** Declares 12 distinct energy smell detection rules backed by academic software engineering research across 5 categories.
- **Radon Complexity Metrics:** Calculates Cyclomatic Complexity, Maximum Nesting Depth, and Function Density to derive a normalized Structural Complexity Index (SCI in [0.0, 1.0]).
- **RAPL Hardware Profiling & Energy Modeling:** Models runtime energy consumption footprint in Joules and kWh using TDP power reference databases.
- **Carbon Intensity Provider Abstraction:** Features a live Electricity Maps API v4 provider and an offline diurnal sine-simulated grid provider for regional grid carbon intensities (gCO₂eq/kWh).
- **Carbon-Aware Recommendation Engine & AST Code Transformer:** Automatically hoists loop-invariant computations, allocations, and context managers outside loops at the AST level.
- **Sandboxed Empirical Runtime Executor:** Executes original and refactored code in isolated subprocesses using high-resolution timing and tracemalloc to record empirical savings.
- **FastAPI Backend Server:** Unified OpenAPI server providing REST endpoints (/analyze, /zones, /search-zones, /forecast, /health).
- **React Dashboard:** Dark-themed HSL glassmorphism visualizer with interactive file upload, zone search, Recharts radar/bar/line charts, and code refactoring diff preview.

2. System Architecture & Pipeline Workflow

The system operates through an end-to-end predictive software carbon estimation pipeline linking static AST parsing, rule evaluation, complexity metrics, hardware power profiling, live regional carbon grid intensity lookups, and automated refactoring.

Stage	Component Module	Functionality & Key Outputs
1. AST Analysis	Static Analysis Engine ('src/analysis')	Parses Python source code into AST; extracts loop nesting, I/O calls, recursion, and allocations.

2. Complexity	Radon Adapter & SCI (`src/complexity`)	Computes Cyclomatic Complexity (CC), Nesting Depth (ND), and Structural Complexity Index (SCI).
3. Smell Detection	Energy Knowledge Base (`src/detector`)	Evaluates 12 EKB rules; assigns confidence c_f per finding and outputs EnergySmellReport.
4. Hardware RAPL	Hardware Profiling Engine (`src/hardware_profile`)	Models TDP reference power (CPU/RAM/Disk) and predicts runtime energy consumption in Joules & kWh.
5. Carbon Grid	Carbon Provider Factory (`src/carbon`)	Queries Electricity Maps API v4 for real-time marginal intensity (gCO_2eq/kWh) with diurnal mock fallback.
6. Sustainability Math	ESS & CIRS Metrics (`src/sustainability`)	Computes Energy Smell Score (ESS) via fuzzy set belief union and physical Carbon Impact Risk Score (CIRS).
7. Refactoring	AST Code Transformer (`src/recommendation`)	Safely hoists loop-invariant computations, allocations, and open file context managers outside loops.
8. Benchmarking	Sandboxed Executor (`src/energy/sandbox.py`)	Runs original vs. optimized code in worker subprocesses measuring empirical time, memory, and energy.

3. Deep-Dive: System Core Engine Subsystems

Static Analysis Engine (`src/analysis/`)

- File: `src/analysis/parser/engine.py`, `src/analysis/visitors/`
- Implements custom AST visitor classes (`LoopVisitor`, `RecursionVisitor`, `MemoryVisitor`, `IOVisitor`) to parse Python code structural constructs without executing arbitrary user scripts.
- Outputs detailed metadata including loop body depth, variable reassignment targets, file read/write operations, and recursive call stacks.

Structural Complexity & Radon Adapter (`src/complexity/`)

- File: `src/complexity/engine.py`, `src/complexity/radon\_adapter.py`, `src/complexity/scorer.py`
- Adapts Radon metrics to calculate Cyclomatic Complexity (CC), Maximum Nesting Depth (ND), and Function Density (FD).
- Normalizes structural metrics into a unified Structural Complexity Index ($SCI \in [0.0, 1.0]$) representing structural algorithmic overhead.

Energy Knowledge Base (EKB) & Rules (`src/knowledge/`, `src/detector/`)

- File: `src/knowledge/rules/energy\_rules.yaml`, `src/detector/engine.py`
- Declares 12 research-backed energy smell rules: Computation (`EKB-COMP`), I/O Operations (`EKB-IO`), Memory Overhead (`EKB-MEM`), Network (`EKB-NET`), and Recursion (`EKB-REC`).
- Confidence Scorer evaluates candidate context and computes finding detection confidence $c_f \in [0.0, 1.0]$.

Hardware Power Profiling & Energy Engine (`src/hardware\_profile/`, `src/energy/`)

- File: `src/hardware\_profile/power.py`, `src/energy/sandbox.py`
- Models baseline Thermal Design Power (TDP) reference power curves for CPU, RAM, and Disk hardware.
- Predicts runtime energy consumption in Joules ($\$J\$$) and kilowatt-hours ($\$kWh\$$).

Carbon Grid Intensity Abstraction (`src/carbon/`)

- File: `src/carbon/providers.py`, `src/carbon/api.py`, `src/carbon/engine.py`

- Provides `ElectricityMapsProvider` querying Electricity Maps API v4 for live marginal carbon grid intensity (\$gCO₂eq/kWh) by ISO country code/zone (e.g. DK-DK1, IN, FR, US-NW).
- Provides `MockCarbonIntensityProvider` simulating diurnal grid fluctuations offline when API keys are absent.

Sustainability Metrics Engine (`src/sustainability/`)

- File: `src/sustainability/metrics.py`
- Computes research-backed Energy Smell Score (ESS) and Carbon Impact Risk Score (CIRS).

Carbon-Aware Recommendation & AST Code Transformer (`src/recommendation/`)

- File: `src/recommendation/engine.py`, `src/recommendation/transformer.py`
- Generates prioritized refactoring advice weighted by carbon risk impact.
- AST Code Transformer parses AST nodes and hoists loop-invariant assignments, allocations, and open file context managers outside loops automatically.

Sandboxed Execution & Empirical Measurement (`src/energy/sandbox.py`)

- File: `src/energy/sandbox.py`
- Runs original vs. optimized code in isolated worker subprocesses with wall-clock timing, process CPU time tracking, and tracemalloc memory profiling.
- Calculates empirical wall-clock and energy reduction percentages.

FastAPI Backend Server (`src/api/`)

- File: `src/api/main.py`, `src/api/models.py`
- Provides CORS-enabled FastAPI REST endpoints (`/analyze`, `/zones`, `/search-zones`, `/forecast`, `/health`).

React HSL Dark Glassmorphism Visualizer (`dashboard/`)

- File: `dashboard/src/App.tsx`, `dashboard/src/index.css`
- Dark-themed visualizer built with React, Vite, TypeScript, Lucide Icons, and Recharts.

4. Mathematical Formulations (ESS & CIRS)

Code-Carbon uses two primary sustainability metrics to quantify software carbon impact:

1. Energy Smell Score (ESS) — Fuzzy Belief Union:

$$ESS = 10.0 \times P_{leak} \quad \text{where} \quad P_{leak} = 1.0 - \prod_{f \in F} (1.0 - c_f)$$

- c_f : Detection confidence of smell finding f (0.0 to 1.0).
- P_{leak} : Fuzzy joint possibility of at least one active energy leak in source code.
- ESS : Scaled continuous score bounded in $[0.0, 10.0]$.

2. Carbon Impact Risk Score (CIRS) — Physical Hazard × Exposure Model:

$$\text{Hazard (H)} = E_{kWh} \times I_{carbon} \quad (\text{Physical carbon emissions per run in gCO}_2\text{eq})$$

$$\text{Exposure (Ex)} = SCI \times (1.0 + (ESS / 10.0)) \quad (\text{Structural complexity \& smell multiplier})$$

$$CIRS = H \times Ex \quad (\text{Effective gCO}_2\text{eq risk score per run})$$

- E_{kWh} : Predicted energy consumption in kWh (Joules / 3.6e6).
- I_{carbon} : Marginal carbon intensity in gCO₂eq/kWh.
- SCI : Structural Complexity Index (0.0 to 1.0).

5. Automated AST Code Transformer & Refactoring Example

The AST Code Transformer (``src/recommendation/transformer.py``) analyzes Python AST nodes and hoists invariant expressions and file context managers outside loops.

Original Unoptimized Code	AST Auto-Transformed Code
<pre>for i in range(1000): with open('data.txt', 'a') as f: x = 10 * 20 f.write(str(x + i))</pre>	<pre>x = 10 * 20 # Hoisted EKB-COMP-002 with open('data.txt', 'a') as f: # Hoisted EKB-I0-001 for i in range(1000): f.write(str(x + i))</pre>

6. REST API Endpoint Specification

HTTP Method	Endpoint Path	Description & Output Schema
GET	/health	Health check endpoint returning <code>`{"status": "healthy"}`</code> .
GET	/zones	Retrieves all supported Electricity Maps grid zones and fallback mock zones.
GET	/search-zones?q=...	Searches supported grid zones dynamically by query string.
POST	/analyze	Uploads source file; returns AST analysis, ESS/CIRS metrics, auto-fixes, and sandboxed run timings.
GET	/forecast?zone=...&joules;=...	Calculates 24-hour diurnal carbon intensity forecast and lowest emission window schedule.

7. Test Suite Verification & Execution Commands

The system includes 92 unit and mathematical tests covering AST visitors, EKB rules, Radon adapters, carbon API providers, sustainability equations, AST transformations, sandboxed runtimes, and FastAPI routers.

OPERATIONAL SHELL COMMANDS:

```
# 1. Run Complete 92-Test Suite
$env:PYTHONPATH="src"; python -m unittest discover -s tests -v

# 2. Launch FastAPI Application Backend Server
$env:PYTHONPATH="src"; uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload

# 3. Launch React Dashboard Frontend
cd dashboard; npm run dev

# 4. CLI Carbon Footprint Estimation
$env:PYTHONPATH="src"; python -m carbon.cli estimate examples/benchmarks/nested_loops.py --zone DK-DK1
```

8. Project Status Summary

SUMMARY:

- **Backend Framework:** Fully functional Python static analysis, energy modeling, live Electricity Maps carbon provider integration, and ESS/CIRS sustainability metrics engines.
- **AST Refactoring:** Automated loop-invariant and I/O context manager hoisting.
- **Empirical Benchmarking:** Isolated sandboxed execution subprocesses measuring wall-clock time and memory savings.
- **Frontend Dashboard:** Modern dark-themed HSL glassmorphism visualizer with interactive charts, grid search, and code diff comparisons.
- **Test Coverage:** 92/92 tests passing cleanly.